

Paradigmas y Lenguajes de Programación III

AA 27 - Accesibilidad, Usabilidad y Seguridad en Aplicaciones Web

Contexto

Esta actividad integra contenidos sobre **estándares de accesibilidad y usabilidad**, la **seguridad en aplicaciones web** y las **normas ISO** que guían la calidad en los sistemas de información.

A través de la observación de los videos, el análisis de amenazas y la revisión crítica de aplicaciones, pondrán poner en práctica conceptos de **W3C**, **OWASP** e **ISO 9126**.

Objetivos

Al finalizar la actividad, deberán poder:

- Comprender los **principios de accesibilidad y usabilidad web** según W3C.
- Reconocer y clasificar **vulnerabilidades y amenazas** en sistemas web.
- Evaluar aplicaciones en función de **estándares ISO y buenas prácticas de seguridad**.
- Proponer **mejoras concretas** para lograr un sistema accesible, usable y seguro.

Materiales y Recursos

1. Videos

-  [Introducción a la Accesibilidad Web – W3C \(Español\)](#)
-  [¿Qué es OWASP Top 10? \(Explicado en 8 minutos\)](#)
-  [Clase 1 – Testing y Accesibilidad – WCAG y ejemplos prácticos](#)
 - Este video, mirar desde el minuto 20 al 60.
-  [TOP 10 OWASP 2021 español con ejemplos – Fallos de Control de Acceso](#)

2. Lecturas

-  [Introducción a la Accesibilidad Web – W3C/WAI](#)
-  [OWASP Top 10 – Riesgos de Seguridad en Aplicaciones Web](#)
-  Norma ISO 9126 – Modelo de Calidad del Software
 - Deberán revisar qué paso con esta norma :)

Paradigmas y Lenguajes de Programación III

Consigna General

A partir de los materiales anteriores:

- **Analizá los videos y lecturas.**
Identifica los principales conceptos de accesibilidad, usabilidad y seguridad.
- **Documenta amenazas a los entornos web y de sistemas.**
Clasificarlas según su tipo:
 - Ataques Cibernéticos
 - Desastres Naturales
 - Errores Humanos
 - Vulnerabilidades Técnicas
- **Elabora un Cuadro Comparativo**
Donde contrastes los distintos tipos de amenazas: causas, consecuencias y controles preventivos.
(Pueden hacerlo en Word, Excel, Canva o similar y exportarlo en **PDF**).
- **Revisá tu aplicación del trabajo final integrador.**
 - Verificá accesibilidad (contraste, teclado, texto alternativo).
 - Evalúa la usabilidad (fluidez, consistencia, claridad).
 - Detecta posibles fallos de seguridad (entradas sin validar, errores de autenticación, etc.).
 - Propone al menos **dos mejoras concretas**.
- **Conclusión individual.**
Reflexiona brevemente (máx. 250 palabras) sobre:
 - ¿Cómo se relacionan la accesibilidad, la usabilidad y la seguridad en una aplicación?
 - ¿Qué norma o estándar te parece más relevante y por qué?

Entrega en el Aula Virtual

Subí un **único archivo en formato PDF** con el siguiente contenido:

1. Portada con tu nombre, comisión y fecha.
2. Cuadro comparativo de amenazas.
3. Evaluación de tu aplicación (con las 2 mejoras propuestas).
4. Conclusión individual.

Paradigmas y Lenguajes de Programación III

DESARROLLO

 **Informe: Accesibilidad, Usabilidad y Seguridad en Aplicaciones Web**

Proyecto: PIN Graphics - Tienda de Tarjetas Gráficas

CUADRO COMPARATIVO DE AMENAZAS

Tipo de Amenaza	Causas	Consecuencias	Controles Preventivos
Ataques Cibernéticos	- Inyección SQL - Cross-Site Scripting (XSS) - Fuerza bruta en login - Robo de sesiones - Ataques DDoS	- Robo de datos de clientes - Modificación de precios - Acceso no autorizado al panel admin - Pérdida de confianza - Indisponibilidad del servicio	- Usar PDO con consultas preparadas - Validar y sanitizar todas las entradas - Implementar CAPTCHA - Usar HTTPS con certificado SSL - Limitar intentos de login - Tokens CSRF en formularios
Desastres Naturales	- Terremotos - Inundaciones - Incendios - Cortes de energía prolongados - Fallas de infraestructura	- Pérdida total de datos - Indisponibilidad del sitio - Pérdida de pedidos en proceso - Imposibilidad de operar	- Backups automáticos diarios - Servidores en múltiples ubicaciones - Cloud hosting con redundancia - Plan de recuperación de desastres

Paradigmas y Lenguajes de Programación III

Tipo de Amenaza	Causas	Consecuencias	Controles Preventivos
Errores Humanos	- Contraseñas débiles o compartidas - Eliminación accidental de productos - Configuración incorrecta - No cerrar sesión en PC públicas - Divulgación de credenciales	- Pérdida de información - Brechas de seguridad - Datos de clientes expuestos - Modificaciones no autorizadas - Acceso no autorizado	Generadores de respaldo - Capacitación continua - Políticas de contraseñas fuertes - Confirmaciones para acciones críticas - Sistema de permisos por roles - Auditoría de acciones - Timeout de sesión automático
	- Software desactualizado - Dependencias con bugs - Configuración insegura - Falta de validaciones - Permisos de archivos incorrectos	- Explotación de vulnerabilidades conocidas - Acceso a archivos sensibles - Escalada de privilegios - Compromiso del servidor	- Actualizar PHP, MySQL, jQuery - Escaneo de vulnerabilidades regular - Configurar permisos de archivos - Ocultar versiones de software - WAF (Web Application Firewall) - Monitoreo de logs

Paradigmas y Lenguajes de Programación III

A. ACCESIBILIDAD (WCAG 2.1)

Aspectos Positivos:

1. **Estructura semántica:** Uso correcto de <header>, <main>, <footer>, <nav>
2. **Atributos ARIA:** aria-label en botones y formularios
3. **Skip links:** Saltar al contenido principal
4. **Labels asociados:** Todos los inputs tienen <label> con for correcto

Problemas Detectados:

1. **Contraste de colores insuficiente:** El texto en var(--muted) podría no cumplir WCAG AA (4.5:1)
 2. **Imágenes sin texto alternativo:** - alt vacío
 3. **Navegación por teclado:** El carrusel Slick podría no ser completamente accesible con Tab
 4. **Foco visible:** No hay estilos :focus visibles para navegación por teclado
 5. **Idioma de página:** Aunque está lang="es-AR", algunos textos en inglés no están marcados
-

B. USABILIDAD (ISO 9241-11)

Aspectos Positivos:

1. **Consistencia visual:** Mismo header/footer en todas las páginas
2. **Feedback visual:** Mensajes de error/éxito en registro/login
3. **Buscador en tiempo real:** Filtrado instantáneo en liveFilter()
4. **Confirmaciones:** Diálogo de confirmación antes de eliminar productos
5. **Responsive:** Media queries para móviles

Problemas Detectados:

1. **Falta de breadcrumbs:** El usuario no sabe dónde está en la jerarquía
2. **Sin indicador de carga:** No hay spinners mientras se cargan productos desde la API

Paradigmas y Lenguajes de Programación III

3. **Carrito sin persistencia:** Al recargar la página, el carrito se pierde
 4. **Errores genéricos:** Mensajes como "Error 1" no son descriptivos
 5. **Sin paginación:** Si hay muchos productos, la tabla puede ser muy larga
 6. **Campo de búsqueda confuso:** Está en el header pero no está claro qué busca
-

C. SEGURIDAD (OWASP TOP 10 2021)

Aspectos Positivos:

1. **Contraseñas hasheadas:** Uso de password_hash() y password_verify()
2. **Consultas preparadas:** PDO con placeholders ? previene SQL Injection
3. **Verificación de sesión:** session_start() y validación de \$_SESSION
4. **Separación de roles:** Admin vs Cliente con permisos diferentes

VULNERABILIDADES CRÍTICAS DETECTADAS:

1. A01:2021 – Broken Access Control (Control de Acceso Roto)

php

// admin.php - Línea 7-10

```
if (!isset($_SESSION['admin_logged_in']) || $_SESSION['admin_logged_in'] !== true) {  
    header('Location: login.html');  
    exit;  
}
```

Problema: Un usuario podría manipular la sesión o acceder directamente a api.php sin autenticación.

2. A03:2021 – Injection (Inyección SQL)

php

// api.php - Línea 25

```
$stmt = $pdo->query("SELECT * FROM productos");
```

Paradigmas y Lenguajes de Programación III

Problema: Aunque no hay entrada del usuario aquí, otros archivos como admin.php reciben datos vía \$_GET['id'] que podrían ser manipulados.

3. A05:2021 – Security Misconfiguration (Configuración Incorrecta)

php

// Múltiples archivos

`$db_pass = ""; // ¡Contraseña vacía en producción!`

Problema: Credenciales hardcodeadas y sin encriptar.

4. A07:2021 – Identification and Authentication Failures

php

// login.php

// No hay límite de intentos de login

// No hay CAPTCHA

// No hay autenticación de dos factores (2FA)

5. A04:2021 – Insecure Design (Diseño Inseguro)

- No hay validación de tipos de archivo en uploads (si se implementara foto de perfil)
- No hay sanitización de salida en admin.php línea 163:

php

`userName.innerHTML = `Hola, ${data.nombre} <span class="badge"...``

Problema: Vulnerable a XSS si el nombre contiene código JavaScript.

6. Fallos adicionales:

- **No hay HTTPS obligatorio:** Las contraseñas viajan en texto plano
- **No hay tokens CSRF:** Los formularios son vulnerables a ataques CSRF
- **Sesiones sin timeout:** No hay expiración automática
- **Sin rate limiting:** Posible ataque de fuerza bruta
- **Archivos sensibles expuestos:** test_db.php muestra información de la base de datos

Paradigmas y Lenguajes de Programación III

PROPUESTAS DE MEJORA

MEJORA 1: Implementar Sistema de Seguridad Robusto

Problema:

Múltiples vulnerabilidades críticas de OWASP TOP 10.

Solución:

A. Agregar protección CSRF en todos los formularios:

php

// Crear archivo: csrf_token.php

<?php

session_start();

```
function generateCSRFToken() {
```

```
    if (!isset($_SESSION['csrf_token'])) {
```

```
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
```

```
    }
```

```
    return $_SESSION['csrf_token'];
```

```
}
```

```
function validateCSRFToken($token) {
```

```
    return isset($_SESSION['csrf_token']) && hash_equals($_SESSION['csrf_token'],  
$token);
```

```
}
```

```
?>
```

Modificar admin.php:

Paradigmas y Lenguajes de Programación III

html

```
<form method="POST" action="admin.php">

  <input type="hidden" name="csrf_token" value="<?php echo generateCSRFToken(); ?>">

  <!-- resto del formulario -->

</form>
```

php

// Al procesar el POST

```
if($_SERVER['REQUEST_METHOD'] === 'POST') {

    if (!isset($_POST['csrf_token']) || !validateCSRFToken($_POST['csrf_token'])) {

        die('Token CSRF inválido');

    }

    // continuar con la lógica

}
```

B. Implementar rate limiting para login:

php

// Crear archivo: rate_limiter.php

```
<?php

function checkRateLimit($identifier, $max_attempts = 5, $time_window = 300) {

    session_start();

    $key = 'login_attempts_' . $identifier;

    if (!isset($_SESSION[$key])) {

        $_SESSION[$key] = ['count' => 0, 'first_attempt' => time()];

    }
```

Paradigmas y Lenguajes de Programación III

```
$attempts = &$_SESSION[$key];
```

```
// Reset si pasó el tiempo
```

```
if (time() - $attempts['first_attempt'] > $time_window) {
```

```
    $attempts = ['count' => 0, 'first_attempt' => time()];
```

```
}
```

```
if ($attempts['count'] >= $max_attempts) {
```

```
    $remaining = $time_window - (time() - $attempts['first_attempt']);
```

```
    return ['allowed' => false, 'wait' => $remaining];
```

```
}
```

```
return ['allowed' => true];
```

```
}
```

```
function incrementAttempts($identifier) {
```

```
    $key = 'login_attempts_' . $identifier;
```

```
    $_SESSION[$key]['count']++;
```

```
}
```

```
?>
```

Modificar login.php:

```
php
```

```
require_once 'rate_limiter.php';
```

```
$email_or_username = trim($_POST['email'] ?? '');
```

Paradigmas y Lenguajes de Programación III

```
$rate_check = checkRateLimit($email_or_username);
```

```
if (!$rate_check['allowed']) {  
  
    header('Location: login.html?error=rate_limit&wait=' . $rate_check['wait']);  
  
    exit;  
  
}
```

```
// ... resto del código de login
```

```
// Si falla el login
```

```
incrementAttempts($email_or_username);
```

C. Sanitizar salidas HTML (prevenir XSS):

```
javascript
```

```
// Modificar app.js - Función updateSessionUI()
```

```
if (userName) {
```

```
    // ANTES (vulnerable):
```

```
    // userName.innerHTML = `Hola, ${data.nombre} <span class="badge"...`;
```

```
    // DESPUÉS (seguro):
```

```
    const sanitizedNombre = data.nombre.replace(/<|>/g, "&lt;");
```

```
    userName.innerHTML = `Hola, ${sanitizedNombre} <span class="badge"...`;
```

```
}
```

D. Mover credenciales a archivo de configuración externo:

```
php
```

```
// Crear archivo: config.php (fuera de public_html)
```

Paradigmas y Lenguajes de Programación III

```
<?php
```

```
return [
```

```
    'db_host' => getenv('DB_HOST') ?: 'localhost',
```

```
    'db_name' => getenv('DB_NAME') ?: 'pin_graphics',
```

```
    'db_user' => getenv('DB_USER') ?: 'root',
```

```
    'db_pass' => getenv('DB_PASS') ?: '',
```

```
];
```

```
?>
```

```
php
```

```
// Modificar todos los archivos PHP con conexión
```

```
$config = require_once '../config.php';
```

```
$pdo = new PDO(
```

```
    "mysql:host={$config['db_host']};dbname={$config['db_name']};charset=utf8mb4",
```

```
    $config['db_user'],
```

```
    $config['db_pass']
```

```
);
```

E. Agregar timeout de sesión:

```
php
```

```
// Al inicio de cada archivo protegido
```

```
session_start();
```

```
$timeout = 1800; // 30 minutos
```

```
if (isset($_SESSION['LAST_ACTIVITY']) && (time() - $_SESSION['LAST_ACTIVITY'] > $timeout)) {
```

Paradigmas y Lenguajes de Programación III

```
session_unset();

session_destroy();

header('Location: login.html?error=session');

exit;

}
```

```
$_SESSION['LAST_ACTIVITY'] = time();
```

F. Eliminar archivo de prueba peligroso:

```
bash

# Borrar test_db.php del servidor

rm test_db.php
```

MEJORA 2: Mejorar Accesibilidad y Usabilidad

Problema:

Contraste insuficiente, falta de textos alternativos, sin indicadores de carga, carrito no persistente.

Solución:

A. Mejorar contraste de colores en styles.css:

```
css

/* ANTES */

:root {

    --muted: #64748b;

}

/* DESPUÉS (cumple WCAG AA) */
```

Paradigmas y Lenguajes de Programación III

```
:root {  
  
  --muted: #475569; /* Contraste mejorado */  
  
}  
  
/* Agregar estilos de foco visibles */  
  
a:focus,  
button:focus,  
input:focus,  
select:focus,  
textarea:focus {  
  
  outline: 3px solid var(--brand);  
  outline-offset: 2px;  
  
}  
  
/* Mejorar contraste en modo oscuro */  
  
[data-theme="dark"] {  
  
  --muted: #94a3b8; /* Más claro en tema oscuro */  
  
}
```

B. Agregar textos alternativos descriptivos:

```
javascript  
  
// Modificar app.js - renderProductPage()  
  
$img.src = data.img || 'img/placeholder.png';  
  
$img.alt = `Imagen de ${data.name}`; // AGREGAR ESTA LÍNEA  
  
// En listado_box y carousel
```

Paradigmas y Lenguajes de Programación III

div.innerHTML = `

```
<article class="card">
```

```

```

```
...
```

```
</article>
```

```
`;
```

C. Agregar indicador de carga:

html

```
<!-- Agregar al final de index.html (antes de </body>) -->
```

```
<div id="loading-overlay" style="display:none;">
```

```
<div class="spinner"></div>
```

```
</div>
```

css

```
/* Agregar a styles.css */
```

```
#loading-overlay {
```

```
position: fixed;
```

```
top: 0;
```

```
left: 0;
```

```
width: 100%;
```

```
height: 100%;
```

```
background: rgba(0, 0, 0, 0.7);
```

```
z-index: 9999;
```

```
display: flex;
```

Paradigmas y Lenguajes de Programación III

```
align-items: center;
```

```
justify-content: center;
```

```
}
```

```
.spinner {
```

```
width: 50px;
```

```
height: 50px;
```

```
border: 5px solid #f3f3f3;
```

```
border-top: 5px solid var(--brand);
```

```
border-radius: 50%;
```

```
animation: spin 1s linear infinite;
```

```
}
```

```
@keyframes spin {
```

```
0% { transform: rotate(0deg); }
```

```
100% { transform: rotate(360deg); }
```

```
}
```

```
javascript
```

```
// Modificar app.js - función getCatalog()
```

```
async function getCatalog() {
```

```
// Mostrar loading
```

```
const loading = document.getElementById('loading-overlay');
```

```
if (loading) loading.style.display = 'flex';
```

```
if (!catalogPromise) {
```

Paradigmas y Lenguajes de Programación III

```
catalogPromise = fetch(API_URL)

.then(response => {

  if (!response.ok) {

    throw new Error(`Error HTTP! status: ${response.status}`);

  }

  return response.json();

})

.catch(error => {

  console.error("Error al cargar el catálogo:", error);

  catalogPromise = null;

  return {};

})

.finally(() => {

  // Ocultar loading

  if (loading) loading.style.display = 'none';

});

}

return catalogPromise;

}
```

D. Implementar persistencia del carrito con localStorage:

javascript

// Agregar al final de app.js

// Función para guardar carrito

```
function saveCart(cart) {
```

Paradigmas y Lenguajes de Programación III

```
try {  
  
    localStorage.setItem('pin_graphics_cart', JSON.stringify(cart));  
  
} catch (e) {  
  
    console.error('Error al guardar carrito:', e);  
  
}  
  
}
```

// Función para cargar carrito

```
function loadCart() {  
  
    try {  
  
        const saved = localStorage.getItem('pin_graphics_cart');  
  
        return saved ? JSON.parse(saved) : [];  
  
    } catch (e) {  
  
        console.error('Error al cargar carrito:', e);  
  
        return [];  
  
    }  
  
}
```

// Modificar CART_KEY para usar localStorage

```
let CART = loadCart();
```

// Modificar función saveToLocalStorage()

```
function saveToLocalStorage() {  
  
    saveCart(CART);  
  
}
```

Paradigmas y Lenguajes de Programación III

// Llamar saveCart() cada vez que se modifica el carrito

```
function addToCart(id, qty = 1) {
```

```
    // ... código existente
```

```
    saveCart(CART);
```

```
    updateCartUI();
```

```
}
```

```
function removeFromCart(id) {
```

```
    // ... código existente
```

```
    saveCart(CART);
```

```
    updateCartUI();
```

```
}
```

```
function emptyCart() {
```

```
    CART = [];
```

```
    saveCart(CART);
```

```
    updateCartUI();
```

```
}
```

E. Mejorar mensajes de error:

php

// Modificar login.php

// ANTES:

```
header('Location: login.html?error=1');
```

Paradigmas y Lenguajes de Programación III

// DESPUÉS:

```
header('Location: login.html?error=invalid_credentials&msg=' . urlencode('Email o contraseña incorrectos'));
```

```
javascript
```

```
// Modificar login.html <script>
```

```
if (params.has('error')) {
```

```
    const errorCode = params.get('error');
```

```
    const customMsg = params.get('msg');
```

```
    let msg = "";
```

```
    switch(errorCode) {
```

```
        case 'invalid_credentials':
```

```
            msg = decodeURIComponent(customMsg) || 'Email/usuario o contraseña incorrectos.';
```

```
            break;
```

```
        case 'rate_limit':
```

```
            const wait = params.get('wait');
```

```
            msg = `Demasiados intentos. Espera ${Math.ceil(wait/60)} minutos.`;
```

```
            break;
```

```
        case 'db_error':
```

```
            msg = 'Error de conexión. Intenta nuevamente.';
```

```
            break;
```

```
        case 'session_expired':
```

```
            msg = 'Tu sesión expiró. Por favor, ingresá nuevamente.';
```

```
            break;
```

Paradigmas y Lenguajes de Programación III

default:

```
msg = 'Ocurrió un error. Intenta nuevamente.';

}
```

```
msgBox.textContent = msg;

msgBox.style.display = 'block';

}
```

F. Agregar breadcrumbs:

html

```
<!-- Agregar después de <header> en todas las páginas -->

<nav class="breadcrumbs" aria-label="Ruta de navegación">

  <ol>

    <li><a href="index.html">Inicio</a></li>

    <li><a href="listado_box.html">Productos</a></li>

    <li aria-current="page">RTX 4080 Super</li>

  </ol>

</nav>
```

css

```
/* Agregar a styles.css */

.breadcrumbs {

  background: var(--card);

  padding: 0.75rem 0;

  border-bottom: 1px solid var(--border);

}
```

Paradigmas y Lenguajes de Programación III

```
.breadcrumbs ol {
```

```
    display: flex;
```

```
    flex-wrap: wrap;
```

```
    list-style: none;
```

```
    padding: 0;
```

```
    margin: 0;
```

```
    gap: 0.5rem;
```

```
}
```

```
.breadcrumbs li:not(:last-child)::after {
```

```
    content: ">";
```

```
    margin-left: 0.5rem;
```

```
    color: var(--muted);
```

```
}
```

```
.breadcrumbs a {
```

```
    color: var(--brand);
```

```
    text-decoration: none;
```

```
}
```

```
.breadcrumbs a:hover {
```

```
    text-decoration: underline;
```

```
}
```

```
.breadcrumbs [aria-current="page"] {
```

Paradigmas y Lenguajes de Programación III

```
color: var(--ink);
```

```
font-weight: 600;
```

```
}
```

G. Hacer el carrusel más accesible:

```
javascript
```

```
// Modificar configuración de Slick en app.js
```

```
$(featuredCarousel).slick({
```

```
  dots: true,
```

```
  infinite: true,
```

```
  speed: 500,
```

```
  slidesToShow: 3,
```

```
  slidesToScroll: 1,
```

```
  autoplay: true,
```

```
  autoplaySpeed: 3000,
```

```
  pauseOnHover: true,
```

```
  pauseOnFocus: true, // AGREGAR: pausa con Tab
```

```
  arrows: true,
```

```
  accessibility: true, // AGREGAR: mejora accesibilidad
```

```
  adaptiveHeight: true,
```

```
// Personalizar textos ARIA
```

```
  prevArrow: '<button type="button" class="slick-prev" aria-label="Producto anterior">Anterior</button>',
```

```
  nextArrow: '<button type="button" class="slick-next" aria-label="Producto siguiente">Siguiendo</button>',
```

Paradigmas y Lenguajes de Programación III

responsive: [...]

});

CONCLUSIÓN INDIVIDUAL

La accesibilidad, usabilidad y seguridad en PIN Graphics están intrínsecamente relacionadas y son fundamentales para el éxito del proyecto. Una aplicación accesible que no valida correctamente las entradas puede ser vulnerable a ataques XSS que afecten a usuarios con lectores de pantalla. Del mismo modo, una interfaz usable pero insegura expone a los clientes a robos de datos.

En mi proyecto, identifiqué que si bien la estructura HTML semántica favorece la accesibilidad, la falta de contraste adecuado y textos alternativos descriptivos limita el acceso a usuarios con discapacidad visual. En términos de usabilidad, aunque la navegación es intuitiva, la ausencia de indicadores de carga y persistencia del carrito genera frustración. Respecto a la seguridad, detecté vulnerabilidades críticas como la falta de protección CSRF, ausencia de rate limiting en el login y sanitización insuficiente de salidas, todas ellas presentes en el OWASP Top 10 2021.

El estándar más relevante para este proyecto es **WCAG 2.1 (Web Content Accessibility Guidelines)** porque establece criterios verificables para hacer el contenido web accesible, lo cual no solo cumple con regulaciones legales sino que amplía el mercado potencial. Además, muchos principios de WCAG (como textos alternativos y contraste) mejoran la usabilidad para todos los usuarios. Sin embargo, OWASP Top 10 es igualmente crítico ya que una brecha de seguridad podría comprometer todos los datos de clientes y destruir la reputación del negocio.

La implementación de las mejoras propuestas transformaría PIN Graphics en una aplicación robusta, inclusiva y segura, cumpliendo con estándares internacionales y protegiendo tanto a los usuarios como al negocio.

Referencias

- W3C Web Accessibility Initiative (WAI) - WCAG 2.1
- OWASP Top Ten 2021
- ISO/IEC 25010:2011 (reemplazó ISO 9126)
- PHP: The Right Way - Security Best Practices

Paradigmas y Lenguajes de Programación III

- MDN Web Docs - Web Accessibility